ATLAS Master Flightline Clock - System Review & Analysis

Document: System Architecture & Gap Analysis
Project: ATLAS Master Flightline Clock
Date: July 3, 2026
Purpose: Foundation document for ICD-001 generation and feature implementation
Analyst: Abacus AI Agent

---

1. Executive Summary

Project ATLAS is a **functional prototype** for a Master Flightline Clock designed for competitive aviation events. The system has evolved from basic JSON configuration to a professional, database-driven appliance architecture running on a Mac Mini M4. The codebase demonstrates **solid architectural separation** between timing logic, persistence, UI, and display rendering.

Current State: v1.5.0 - Mission-Ready Prototype

Key Strengths:
- Clean MVC architecture with modular design
- Hardware-agnostic timing engine
- Adaptive 6-row LED display strategy (128x64 and 64x64 profiles)
- SQLite persistence with audit trail foundation
- Professional operator console with custom navigation

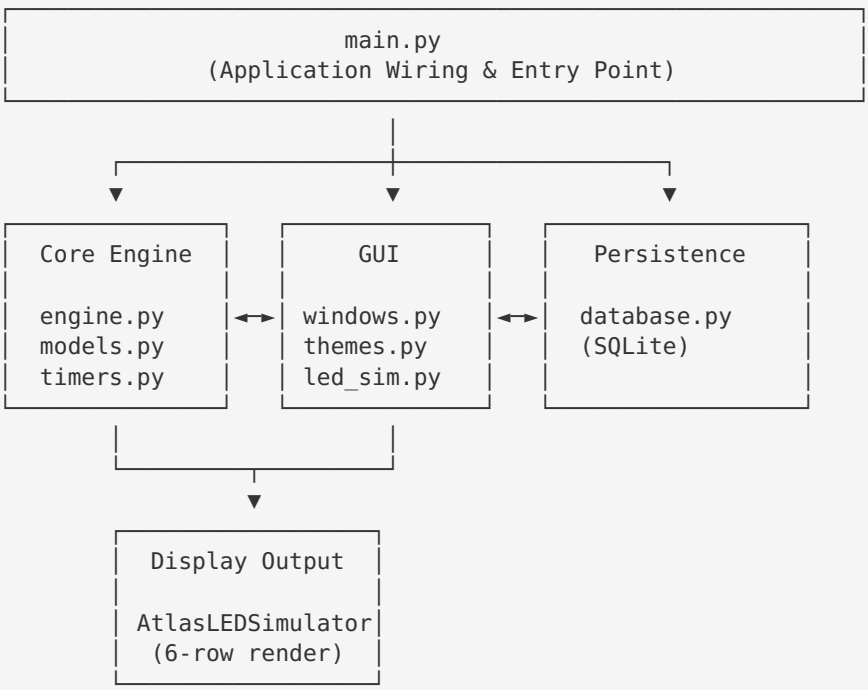
Critical Gaps:
- No physical hardware interface (simulator only)
- Missing Interface Control Document (ICD-001) for LED hardware
- Auto-boot appliance functionality not implemented
- Hardware abstraction layer (HAL) not present

---

2. System Architecture Analysis

2.1 Current Architecture Overview

The system implements a clean **Model-View-Controller (MVC)** pattern with the following layers:



2.2 Component Breakdown

Core Engine (engine.py)

- **Responsibility:** State machine and timing logic
- **Update Rate:** 1 Hz (1000ms ticker)
- **State Model:** Phases P0-P4 based on UTC time comparison
- **Signals:** `model_updated`, `brightness_changed`
- **Independence:** Fully decoupled from UI and display hardware

Key Features:

- Automatic heat activation (15 minutes before track open)
- Phase calculation based on track_open, track_close, heat_end timestamps
- Countdown formatting (MM:SS)
- Heat cancellation logic

State Transition Logic:

```
Phase 0 (ARMED):       now < (track_open - 300s)  → "TRACK OPENS IN"
Phase 1 (PRE-ENTRY):   now > (track_open - 300s)  → [Same as P0]
Phase 2 (TRACK ENTRY): track_open to track_close  → "ENTER NOW" (Green)
Phase 3 (ACTIVE):      track_close to heat_end    → "HEAT TIME REMAINING" (Red)
Phase 4 (COOLDOWN):    now > heat_end             → "HEAT COMPLETED"
```

Persistence Layer (database.py)

- **Technology:** SQLite3 embedded database
- **Location:** `data/atlas.db`
- **Tables:**
- `heats`: heat_number, thermalling_dir, track_open, track_close, heat_end, status
- `settings`: key-value storage (competition_name, display_width, display_height, led_scale, brightness)

CRUD Operations:
- save_heat(), get_todays_schedule(), get_active_heat(), get_next_pending_heat()
- set_heat_status(), cancel_active_heat(), delete_heat()
- save_setting(), get_setting()

Audit Trail Foundation: Heat status transitions (READY → ACTIVE → COMPLETED/CANCELLED) are preserved, not deleted.

Data Models (models.py)

- **HeatConfig:** Pydantic model for heat scheduling data
- **AtlasDisplayModel:** Display state container broadcasted by engine

```
AtlasDisplayModel:
  - local_time: str          # "HH:MM:SS"
  - heat_number: str         # "01", "02", etc.
  - thermalling_dir: str     # "LEFT" or "RIGHT"
  - planned_open: str        # "HH:MM"
  - planned_close: str       # "HH:MM"
  - primary_timer: str       # "MM:SS"
  - primary_timer_label: str # "TRACK OPENS IN", "ENTER NOW", etc.
  - primary_timer_color: str # Hex color "#FFFF00"
  - band_color: str          # Status bar color
  - is_armed: bool           # System active state
```

Operator Console (windows.py)

- **Framework:** PySide6 (Qt6)
- **Style:** Fusion (cross-platform consistency)
- **Navigation:** Custom button bar (QStackedWidget) - replaced native QTabWidget due to macOS styling limitations

Three Tabs:

1. **LIVE Tab:**
   - Real-time heat status display
   - “ABORT / CANCEL ACTIVE HEAT” button
   - Updates via `engine.model_updated` signal

2. **SCHEDULE Tab:**
   - Heat list table (QTableWidget)
   - Add/Edit/Delete heat functionality
   - Form inputs: heat_number, thermalling_dir, track_open, track_close, heat_end
   - Row selection for editing
   - Validation: track_close must be after track_open

3. **SETTINGS Tab: ✅ ALREADY IMPLEMENTED**
   - Competition name input
   - **Display size dropdown:** “128x64 (Large)” / “64x64 (Compact/Half)”
   - LED scale input (simulator pixel size)
   - **Brightness slider:** 10-100%, labeled (Night/Overcast/Normal/Bright Day/Full Sun)
   - Live brightness updates (no restart required)

- “SAVE & APPLY” button
- **System Controls Section:**
  - ✅ “RESTART ATLAS” button (os.execv process replacement)
  - ✅ “EXIT TO MACOS” button (QApplication.quit)

Graceful Shutdown: `closeEvent()` includes placeholder comment for hardware clear frame:

```
# HARDWARE - Send CLEAR/OFFLINE frame to LED panels here before exit
```

LED Simulator (led_simulator.py)

- **Purpose:** Public display simulation (represents physical LED matrix)
- **Rendering:** Custom bitmap font (5x7), circular LED pixels
- **Adaptive Layout:** Automatically selects Full or Compact mode based on display width

6-Row Display Strategy:

Full Mode (128x64):

```
Row 1: Local Time              (Medium, Cyan)
Row 2: Heat Info (H01 LEFT)    (Medium, Yellow)
Row 3: Windows (O 14:30 C 14:35) (Small, Gray)
Row 4: Phase Label              (Small, Phase Color)
Row 5: Primary Timer            (Large 2x, Phase Color)
Row 6: Status Band              (2px height, Phase Color)
```

Compact Mode (64x64):

```
Row 1: Local Time              (Centered, Cyan)
Row 2: Heat + Direction        (Centered, Yellow)
Row 3: Track Open Time         (Centered, Gray)
Row 4: Track Close Time        (Centered, Gray)
Row 5: Phase Label (abbreviated) (Centered, Phase Color)
Row 6: Primary Timer            (Centered, Phase Color)
```

Label Abbreviations (Compact Mode):
- “TRACK OPENS IN” → “OPEN IN”
- “ENTER NOW” → “ENTER”
- “HEAT TIME REMAINING” → “HEAT LEFT”
- “WAITING” → “WAIT”

Brightness Control:
- Live adjustment via slider (10-100%)
- `_apply_brightness()` method scales RGB values
- Immediate repaint without restart
- Emits via `engine.brightness_changed` signal

Supporting Modules

themes.py:
- COCKPIT_BG: Dark background (#0A0A0A)
- BTN_MASTER_ARM: Blue action buttons (#0066CC)
- BTN_CANCEL: Red abort buttons (#880000)
- TABS: Custom tab styling (unused after migration to button navigation)

timers.py:
- AtlasTimerEngine: 10Hz ticker with signal-based countdown
- Currently **unused** in main application (engine.py uses its own 1Hz QTimer)
- Kept for potential future precision timing needs

config.py:
- AtlasConfig: Legacy JSON configuration loader
- **Currently unused** (replaced by database settings)
- Calculates display dimensions from panel counts (P10 panel math: 32px wide, 16px high)
- Kept for reference/future hardware panel configuration

---

3. Documentation Coverage Analysis

3.1 Existing Documentation

Document	Status	Completeness	Quality
atlas_system_handover_dossier_2026-07-03.pdf	✅ Complete	High	Mission-ready handover summary
atlas_technical_update_2026-07-02.docx	✅ Complete	High	Development progress narrative
atlas_sds_001_software_design_spec.pdf	✅ Complete	Medium-High	3 pages, covers timing/persistence/rendering
ATLAS-DEP-001_Deployment_Guide.pdf	🟡 Draft	Medium	Outlines appliance philosophy, needs implementation details
tm_001_traceability_matrix.pdf	🟡 Draft	Low-Medium	Basic requirement mapping, needs test procedures
ATLAS-ICD-001 (Interface Control Document)	❌ Missing	0%	**CRITICAL GAP** - No LED hardware interface spec

3.2 Documentation Quality Assessment

Strengths:
- Professional tone and structure
- Clear architectural principles

- GAMP 5 influence (audit trail, traceability mindset)
- Tonal alignment with user preferences (casual, technical, concise)

Weaknesses:
- ICD-001 completely absent (hardware interface undefined)
- TM-001 has no test procedures or verification methods
- ATLAS-DEP-001 lacks specific launcher scripts and installation steps
- No user manual or operator quick-reference guide
- No hardware commissioning procedures

---

4. Gap Analysis: Requested Features vs. Current State

4.1 Feature Request Assessment

Feature	Status	Implementation Notes
Settings Tab with Display Type Dropdown	✅ **IMPLEMENTED**	Line 172-178 in windows.py: QComboBox with “128x64 (Large)” and “64x64 (Compact/Half)” options. Persists to database.
Auto-boot to Clock	❌ **MISSING**	Mentioned in ATLAS-DEP-001 and TM-001 (URS-009) as “Planned”. No launcher script exists.
“Exit to macOS” Button	✅ **IMPLEMENTED**	Line 234-238 in windows.py: “EXIT TO MACOS” button with QMessageBox confirmation, calls QApplication.quit().
Architectural Separation (Engine vs Display)	✅ **GOOD FOUNDATION**	Engine emits display model via signals. LED simulator is decoupled. Hardware abstraction layer missing.

4.2 Critical Observation: Display Type Ambiguity

The user request mentions **“LED Module” vs “LCD Monitor”** as display types, but the current implementation has:
- “128x64 (Large)” → Implies LED matrix or high-resolution output
- “64x64 (Compact/Half)” → Implies smaller LED matrix

Potential Interpretation Gap:
1. **Option A:** User wants conceptual display output type selector:

- “LED Module” (physical hardware) → then choose resolution (64x64, 128x64, 256x32, etc.)
- “LCD Monitor” (simulated on computer screen) → fullscreen/windowed mode

1. **Option B:** User wants the existing dropdown renamed to clarify intent:
   - “128x64 LED Matrix (Full Layout)”
   - “64x64 LED Matrix (Compact Layout)”

2. **Option C:** User wants both:
   - **Display Type:** LED Module / LCD Monitor (top-level selector)
   - **LED Resolution:** 64x64 / 128x64 / 256x32 (conditional, only if LED Module selected)

Recommendation: Clarify with user whether existing implementation satisfies requirement or if architectural change is needed.

---

5. Technical Specifications for ICD-001

5.1 LED Hardware Context

Based on code comments and configuration patterns, the system is designed for **P5/P10 LED matrix panels**:

P10 Panel Specifications (Standard Outdoor Module):
- **Resolution:** 32×16 pixels per module
- **Pixel Pitch:** 10mm
- **Module Dimensions:** 320mm × 160mm
- **Scan Rate:** 1/8 or 1/16 scan
- **Color Depth:** RGB (16-bit typical)
- **Brightness:** 5000-8000 cd/m² (outdoor rated)

P5 Panel Specifications (Higher Density):
- **Resolution:** 64×32 pixels per module
- **Pixel Pitch:** 5mm
- **Module Dimensions:** 320mm × 160mm
- **Scan Rate:** 1/16 or 1/32 scan

Current Configuration Targets:
- **64×64 Display:** 2×4 P10 modules (2 wide, 4 high) = 640mm × 640mm
- **128×64 Display:** 4×4 P10 modules (4 wide, 4 high) = 1280mm × 640mm

5.2 ICD-001 Required Sections

To generate a comprehensive Interface Control Document, the following information is needed:

5.2.1 Physical Interface Specifications

Required Information:
- [ ] Connector types (HUB75, HUB08, or proprietary)
- [ ] Power requirements (voltage, current per panel, total power budget)
- [ ] Cable specifications (length limits, shielding requirements)
- [ ] Environmental ratings (IP rating, temperature range, UV resistance)

HUB75 Interface (Most Common):

```
Pin Assignment:
R1, G1, B1   - RGB data for upper half
R2, G2, B2   - RGB data for lower half
A, B, C, D   - Row address lines
CLK          - Clock signal
LAT          - Latch signal
OE           - Output enable (brightness control via PWM)
GND          - Ground
```

Electrical Characteristics (Typical):
- Logic Level: 3.3V or 5V TTL
- Clock Frequency: 1-20 MHz
- Current Draw: 2-8A per 32×16 module at full brightness

5.2.2 Communication Protocols

Three Primary Options:

Option 1: Direct GPIO Control (Raspberry Pi)
- Bit-banged HUB75 protocol
- Libraries: rpi-rgb-led-matrix (C++), rgbmatrix (Python wrapper)
- **Pros:** Lowest latency, no external controller
- **Cons:** Requires Raspberry Pi, CPU-intensive, limited panel count

Option 2: Serial/SPI Controller
- Dedicated LED controller board (e.g., Novastar, Linsn, Colorlight)
- Protocol: Proprietary serial or Ethernet-based
- **Pros:** Offloaded processing, supports large arrays
- **Cons:** Requires protocol documentation, more expensive

Option 3: Network-Based (Artnet/E1.31)
- DMX-over-Ethernet protocols
- Controllers: PixLite, ESPixelStick, WLED
- **Pros:** Flexible, long cable runs, standard protocols
- **Cons:** Network latency, overkill for static displays

Current Codebase Assumption: None specified. Simulator is software-only.

5.2.3 Data Format & Timing

Frame Structure (HUB75 Direct Drive):

```
For 64×64 display (4096 pixels):
- Frame Rate Target: 60-120 Hz (16.7-8.3ms per frame)
- Bit Depth: 8-bit per color (24-bit RGB)
- Data Size: 64 × 64 × 3 bytes = 12,288 bytes per frame
- Scan Pattern: Row-by-row interlaced (upper/lower half simultaneously)
```

Timing Requirements:
- **Refresh Rate:** Minimum 60 Hz to avoid flicker
- **Update Latency:** <100ms for countdown updates (1Hz update is acceptable)
- **Brightness PWM:** 100-4000 Hz (perceptual linearity via gamma correction)

5.2.4 Coordinate System & Mapping

Atlas Display Model to Physical Pixels:

```
Atlas Logical Coordinates:
  Origin: Top-left (0, 0)
  X-axis: Left to right (0 to display_width-1)
  Y-axis: Top to bottom (0 to display_height-1)

Panel Chaining (for 128×64 = 4×4 P10 modules):
  [Panel 0,0] [Panel 1,0] [Panel 2,0] [Panel 3,0]
  [Panel 0,1] [Panel 1,1] [Panel 2,1] [Panel 3,1]
  [Panel 0,2] [Panel 1,2] [Panel 2,2] [Panel 3,2]
  [Panel 0,3] [Panel 1,3] [Panel 2,3] [Panel 3,3]

Data Stream Mapping: Serpentine or linear (controller-dependent)
```

Color Space:

- Input: Hex RGB (“#00FFFF” → R=0, G=255, B=255)
- Output: May require gamma correction (typical gamma=2.2 for perceptual linearity)

5.2.5 Hardware Abstraction Layer Design

Proposed Architecture:

```python
# Abstract Base Class
class DisplayBackend(ABC):
    @abstractmethod
    def initialize(self, width, height) -> bool:
        """Initialize hardware/connection"""
        pass

    @abstractmethod
    def render_frame(self, model: AtlasDisplayModel):
        """Render the current display model"""
        pass

    @abstractmethod
    def set_brightness(self, percent: int):
        """Set brightness 0-100%"""
        pass

    @abstractmethod
    def clear(self):
        """Blank the display"""
        pass

    @abstractmethod
    def shutdown(self):
        """Clean hardware shutdown"""
        pass

# Concrete Implementations
class LEDSimulatorBackend(DisplayBackend):
    """Current implementation - software simulation"""
    pass

class HUB75DirectBackend(DisplayBackend):
    """Direct GPIO control for Raspberry Pi"""
    pass

class NetworkBackend(DisplayBackend):
    """Artnet/E1.31 network protocol"""
    pass

class SerialControllerBackend(DisplayBackend):
    """Serial/USB LED controller protocol"""
    pass
```

Backend Selection:

```python
# In main.py or config
backend_type = db.get_setting("display_backend", "simulator")

if backend_type == "simulator":
    display = LEDSimulatorBackend(config)
elif backend_type == "hub75":
    display = HUB75DirectBackend(config)
elif backend_type == "network":
    display = NetworkBackend(config)
# ... etc
```

5.3 ICD-001 Open Questions

To finalize the Interface Control Document, the following must be determined:

1. **Primary Hardware Target:**
   - [ ] Raspberry Pi 4/5 with direct HUB75 control?
   - [ ] Dedicated LED controller (which brand/model)?
   - [ ] Network-based DMX controller?
   - [ ] Custom PCB/FPGA solution?

2. **Panel Specifications:**
   - [ ] P5 or P10 modules?
   - [ ] Indoor or outdoor rated?
   - [ ] Specific supplier/model number?
   - [ ] Power supply specifications?

3. **Control Computer:**
   - [ ] Mac Mini M4 (current) - requires USB/Ethernet adapter
   - [ ] Raspberry Pi 4/5 - native GPIO
   - [ ] Windows PC - USB/serial controller
   - [ ] Dual system (Mac for operator, Pi for display)?

4. **Cable Run:**
   - [ ] Distance from computer to LED panels?
   - [ ] Indoor/outdoor cabling requirements?
   - [ ] Power cable separation from data cables?

5. **Failsafe Behavior:**
   - [ ] Display state on application crash?
   - [ ] Display state on power loss/restore?
   - [ ] Watchdog requirements?

---

6. Code Structure Analysis

6.1 Directory Structure

```
Project_Atlas/
├── src/
│   ├── main.py                (45 lines)  - Entry point, application wiring
│   ├── core/
│   │   ├── models.py          (24 lines)  - Pydantic data models
│   │   ├── database.py        (88 lines)  - SQLite CRUD operations
│   │   └── engine.py          (91 lines)  - State machine & timing logic
│   └── gui/
│       ├── windows.py         (303 lines) - Operator console (3 tabs)
│       ├── themes.py          (49 lines)  - Stylesheet definitions
│       ├── led_simulator.py   (179 lines) - Public display renderer
│       ├── fonts.py           (not provided, referenced in led_simulator.py)
│       └── timers.py          (41 lines)  - Precision timer (currently unused)
├── data/
│   └── atlas.db               (SQLite database - not in upload)
├── docs/
│   ├── atlas_sds_001_software_design_spec.pdf
│   ├── atlas_technical_update_2026-07-02.docx
│   ├── atlas_system_handover_dossier_2026-07-03.pdf
│   ├── ATLAS-DEP-001_Deployment_Guide.pdf
│   └── tm_001_traceability_matrix.pdf
└── (Other files not provided: run_atlas.sh, fonts.py)
```

Total Python Code: ~820 lines (excluding fonts.py and launcher scripts)

6.2 Code Quality Assessment

Strengths:
- ✅ Clean separation of concerns (MVC respected)
- ✅ Pydantic models for type safety
- ✅ Signal-based decoupling (Qt signals/slots)
- ✅ Database abstraction with SQLite row_factory for dict results
- ✅ Adaptive rendering based on display dimensions
- ✅ Graceful shutdown patterns
- ✅ Professional error handling (QMessageBox confirmations)

Weaknesses:
- ⚠️ timers.py implemented but unused (engine.py uses QTimer directly)
- ⚠️ config.py legacy JSON loader still present but unused
- ⚠️ No unit tests or integration tests
- ⚠️ No logging framework (only print statements)
- ⚠️ fonts.py not provided (missing dependency for led_simulator.py)
- ⚠️ No hardware interface abstraction (simulator is tightly coupled)

Technical Debt:
- Remove or refactor timers.py (redundant with engine.py QTimer)
- Remove or repurpose config.py (superseded by database settings)
- Add proper logging (Python `logging` module)
- Create abstract display backend interface
- Add validation tests for phase transition logic

6.3 Dependencies

Identified from Imports:
- PySide6 (Qt6 for Python)
- pydantic
- sqlite3 (standard library)
- time, os, sys (standard library)

Likely Additional Dependencies (not visible in provided code):
- fonts.py with AtlasBitmapFont class (custom module)
- Potentially Pillow/PIL for future image rendering

---

7. Feature Implementation Recommendations

7.1 Settings Tab Enhancement (if needed)

Current State: ✅ Already has display dropdown, brightness slider, restart/exit buttons

If "LED Module vs LCD Monitor" distinction is required:

Option 1: Rename Existing Dropdown

```
self.combo_disp_size.addItems([
    "LED Matrix 128x64 (Full Layout)",
    "LED Matrix 64x64 (Compact Layout)",
    "LCD Monitor Fullscreen"  # <-- New option
])
```

Option 2: Two-Level Selection

```
# Display Type Selector
self.combo_display_type = QComboBox()
self.combo_display_type.addItems(["LED Module", "LCD Monitor"])
self.combo_display_type.currentIndexChanged.connect(self._on_display_type_changed)

# Resolution Selector (conditional visibility)
self.combo_resolution = QComboBox()
self.combo_resolution.addItems(["64x64", "128x64", "256x32"])
self.combo_resolution.setVisible(display_type == "LED Module")
```

Backend Selection:

```
# In settings tab
self.combo_backend = QComboBox()
self.combo_backend.addItems([
    "Simulator (Development)",
    "HUB75 Direct (Raspberry Pi)",
    "Network (Artnet/E1.31)",
    "Serial Controller"
])
```

7.2 Auto-Boot Implementation

macOS LaunchAgent Implementation:

Step 1: Create LaunchAgent plist

```
<!-- ~/Library/LaunchAgents/com.atlas.flightline.plist -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.atlas.flightline</string>

    <key>ProgramArguments</key>
    <array>
        <string>/usr/bin/python3</string>
        <string>/Applications/Atlas/main.py</string>
    </array>

    <key>RunAtLoad</key>
    <true/>

    <key>KeepAlive</key>
    <dict>
        <key>SuccessfulExit</key>
        <false/>
    </dict>

    <key>StandardOutPath</key>
    <string>/tmp/atlas.log</string>

    <key>StandardErrorPath</key>
    <string>/tmp/atlas.error.log</string>
</dict>
</plist>
```

Step 2: Enable Automatic Login

```
# macOS System Preferences
sudo defaults write /Library/Preferences/com.apple.loginwindow autoLoginUser "at-
las_operator"
```

Step 3: Disable Sleep/Screensaver

```
sudo pmset -a displaysleep 0
sudo pmset -a sleep 0
defaults -currentHost write com.apple.screensaver idleTime 0
```

Step 4: Installation Script

```
#!/bin/bash
# install_atlas.sh

echo "Installing ATLAS Master Flightline Clock..."

# Copy application files
sudo mkdir -p /Applications/Atlas
sudo cp -r src/* /Applications/Atlas/

# Install LaunchAgent
cp com.atlas.flightline.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.atlas.flightline.plist

# Configure system
sudo pmset -a displaysleep 0
defaults -currentHost write com.apple.screensaver idleTime 0

echo "Installation complete. Reboot to test auto-start."
```

7.3 Hardware Abstraction Layer Implementation

Step 1: Create Abstract Interface

```python
# display_backend.py
from abc import ABC, abstractmethod
from core.models import AtlasDisplayModel

class DisplayBackend(ABC):
    """Abstract base class for display output backends."""

    @abstractmethod
    def initialize(self, config) -> bool:
        """Initialize display hardware/connection."""
        pass

    @abstractmethod
    def render(self, model: AtlasDisplayModel):
        """Render the current display state."""
        pass

    @abstractmethod
    def set_brightness(self, percent: int):
        """Set display brightness 0-100%."""
        pass

    @abstractmethod
    def clear(self):
        """Clear/blank the display."""
        pass

    @abstractmethod
    def shutdown(self):
        """Graceful shutdown and cleanup."""
        pass
```

Step 2: Refactor Existing Simulator

```python
# led_simulator_backend.py
from display_backend import DisplayBackend
from gui.led_simulator import AtlasLEDSimulator

class SimulatorBackend(DisplayBackend):
    def __init__(self):
        self.window = None

    def initialize(self, config) -> bool:
        self.window = AtlasLEDSimulator(config)
        self.window.show()
        return True

    def render(self, model: AtlasDisplayModel):
        if self.window:
            self.window.update_model(model)

    def set_brightness(self, percent: int):
        if self.window:
            self.window.set_brightness(percent)

    def clear(self):
        # Render empty/black frame
        pass

    def shutdown(self):
        if self.window:
            self.window.close()
```

Step 3: Create Hardware Backend Stub

```python
# led_hardware_backend.py
from display_backend import DisplayBackend

class HUB75Backend(DisplayBackend):
    def __init__(self):
        self.matrix = None

    def initialize(self, config) -> bool:
        try:
            from rgbmatrix import RGBMatrix, RGBMatrixOptions
            options = RGBMatrixOptions()
            options.rows = 16
            options.cols = 32
            options.chain_length = 4  # For 128x64
            options.parallel = 2
            self.matrix = RGBMatrix(options=options)
            return True
        except ImportError:
            print("ERROR: rgbmatrix library not available")
            return False

    def render(self, model: AtlasDisplayModel):
        if not self.matrix:
            return

        # TODO: Render model to matrix
        # self.matrix.Clear()
        # Draw text, shapes to self.matrix
        # Use graphics library to convert AtlasDisplayModel to pixels
        pass

    def set_brightness(self, percent: int):
        if self.matrix:
            self.matrix.brightness = int(percent * 100)

    def clear(self):
        if self.matrix:
            self.matrix.Clear()

    def shutdown(self):
        if self.matrix:
            self.matrix.Clear()
            # HUB75 hardware doesn't need explicit shutdown
```

Step 4: Backend Factory

```python
# backend_factory.py
from display_backend import DisplayBackend
from led_simulator_backend import SimulatorBackend
from led_hardware_backend import HUB75Backend

def create_display_backend(backend_type: str) -> DisplayBackend:
    """Factory function to instantiate display backends."""

    if backend_type == "simulator":
        return SimulatorBackend()
    elif backend_type == "hub75":
        return HUB75Backend()
    elif backend_type == "network":
        # return NetworkBackend()
        raise NotImplementedError("Network backend not yet implemented")
    else:
        raise ValueError(f"Unknown backend type: {backend_type}")
```

Step 5: Update main.py

```python
# main.py (modified)
from backend_factory import create_display_backend

def main():
    app = QApplication(sys.argv)
    app.setStyle(QStyleFactory.create("Fusion"))

    db = AtlasDatabase()
    engine = AtlasEngine(db)
    config = DisplayConfig(db)

    # Backend selection from settings
    backend_type = db.get_setting("display_backend", "simulator")
    display_backend = create_display_backend(backend_type)

    if not display_backend.initialize(config):
        print("ERROR: Failed to initialize display backend")
        sys.exit(1)

    # Connect engine to backend
    engine.model_updated.connect(display_backend.render)
    engine.brightness_changed.connect(display_backend.set_brightness)

    # Launch operator console
    operator_win = OperatorWindow(engine)
    operator_win.show()

    sys.exit(app.exec())
```

7.4 Exit to macOS Button Enhancement

Current Implementation: ✅ Already exists (line 234-238 in windows.py)

Potential Enhancement: Confirmation with Heat Status

```
def _shutdown_app(self):
    active_heat = self.engine.db.get_active_heat()

    if active_heat:
        warning_msg = f"WARNING: Heat {active_heat['heat_number']} is currently ACT-
IVE!\n\n"
        warning_msg += "Shutting down will stop the public display.\n"
        warning_msg += "Are you sure you want to exit?"

        reply = QMessageBox.warning(self, 'Active Heat Warning', warning_msg,
                    QMessageBox.Yes | QMessageBox.No, QMessageBox.No)
    else:
        reply = QMessageBox.question(self, 'System Shutdown',
                    "Are you sure you want to exit Atlas?",
                    QMessageBox.Yes | QMessageBox.No)

    if reply == QMessageBox.Yes:
        # Hardware cleanup before exit
        # TODO: Call display_backend.shutdown()
        from PySide6.QtWidgets import QApplication
        QApplication.quit()
```

---

8. ICD-001 Generation Strategy

8.1 Document Structure Proposal

ATLAS-ICD-001: LED Display Interface Control Document

Sections:

1. **Introduction**
- Purpose and Scope
- Applicable Documents
- Terminology and Abbreviations

1. **System Overview**
   - Architecture Diagram (Mac Mini → Controller → LED Panels)
   - Data Flow (AtlasEngine → DisplayBackend → Physical Hardware)
   - Interface Boundaries

2. **Physical Interface Specifications**
   - Connector Pinouts (HUB75/HUB08)
   - Cable Requirements
   - Power Specifications
   - Mechanical Mounting

3. **Electrical Characteristics**
   - Voltage Levels
   - Current Requirements
   - Signal Timing Diagrams
   - EMC/EMI Considerations

4. **Communication Protocol**
   - Data Frame Structure
   - Timing Requirements

   - Handshaking (if applicable)
   - Error Detection/Correction

5. **Data Mapping**
   - Coordinate System
   - Color Space (RGB bit depth, gamma)
   - Brightness Control (PWM frequency, linear/log mapping)
   - Panel Addressing Scheme

6. **Software Interface**
   - DisplayBackend Abstract Class Specification
   - Backend Implementation Requirements
   - State Machine Integration Points
   - Error Handling Requirements

7. **Performance Requirements**
   - Refresh Rate (minimum 60 Hz)
   - Update Latency (maximum 100ms)
   - Brightness Range (10-100%)
   - Environmental Operating Conditions

8. **Testing & Validation**
   - Interface Compliance Tests
   - Performance Benchmarks
   - Acceptance Criteria

9. **Appendices**
   - Example Code Snippets
   - Vendor Datasheets
   - Troubleshooting Guide

8.2 Information Gathering Needed

To complete ICD-001, the following must be provided by stakeholders:

Hardware Selection:
- [ ] LED panel manufacturer/model
- [ ] Controller board type
- [ ] Computer interface (GPIO, USB, Ethernet)

Installation Environment:
- [ ] Indoor or outdoor installation
- [ ] Viewing distance (affects pixel pitch selection)
- [ ] Ambient light conditions (affects brightness requirements)
- [ ] Power availability (voltage, current capacity)

Performance Requirements:
- [ ] Acceptable refresh rate (Hz)
- [ ] Maximum update latency (ms)
- [ ] Brightness adjustment resolution (10% steps vs. 1% steps)

Regulatory Requirements:
- [ ] Electrical safety standards (UL, CE, SAA)

- [ ] EMC compliance requirements
- [ ] Environmental ratings (IP rating)

8.3 Provisional ICD-001 Approach

Given Current Information Gaps:

Strategy: Generate a **modular ICD-001** with:
- **Section A:** Generic LED matrix interface theory (applicable to any hardware)
- **Section B:** Example implementations (HUB75, Network, Serial)
- **Section C:** Placeholder sections for specific hardware (to be completed after procurement)

Benefit: Provides architectural guidance immediately while allowing hardware-specific details to be added later.

---

9. Implementation Priority & Roadmap

Phase 1: Documentation (Immediate)

1. ✅ Generate ATLAS-ICD-001 (modular/generic version)
2. ✅ Document Hardware Abstraction Layer design
3. Update TM-001 with new requirements (URS-011: HAL, URS-012: Backend selection)

Phase 2: Code Refactoring (Week 1)

1. Create DisplayBackend abstract class
2. Refactor LEDSimulator into SimulatorBackend
3. Add backend selection to Settings tab
4. Update main.py to use backend factory
5. Add logging framework (replace print statements)

Phase 3: Auto-Boot Implementation (Week 1-2)

1. Create LaunchAgent plist for macOS
2. Write installation script (install_atlas.sh)
3. Test automatic startup on reboot
4. Document uninstall procedure

Phase 4: Hardware Integration (Pending Procurement)

1. Acquire LED panels and controller
2. Implement HUB75Backend or SerialControllerBackend
3. Test frame rendering and brightness control
4. Finalize ICD-001 with actual hardware specifications
5. Commissioning and field testing

Phase 5: Production Hardening (Post-Hardware)

1. Add unit tests (pytest for engine logic)
2. Add integration tests (hardware interface validation)
3. Implement watchdog process
4. Create backup/restore functionality
5. Write operator training manual

10. Open Questions & Decisions Required

10.1 User/Stakeholder Decisions

1. **Display Type Terminology:**
   - Is existing “128x64 (Large)” / “64x64 (Compact)” dropdown sufficient?
   - Or is a “LED Module” vs “LCD Monitor” conceptual distinction needed?
   - Should there be a backend selector (Simulator, Hardware, Network)?

2. **Hardware Platform:**
   - Will physical LED panels use Raspberry Pi + HUB75?
   - Or dedicated LED controller (Novastar, Linsn, etc.)?
   - Or network-based DMX/Artnet controller?

3. **Panel Specifications:**
   - P5 (5mm pitch, higher density) or P10 (10mm pitch, standard)?
   - Indoor or outdoor rated?
   - Specific supplier/model number?

4. **ICD-001 Detail Level:**
   - Generate generic/architectural version now, finalize later?
   - Or wait until hardware is procured for complete specifications?

5. **Auto-Boot Priority:**
   - Implement macOS LaunchAgent now (can test with simulator)?
   - Or defer until hardware is ready?

10.2 Technical Decisions

1. **Rendering Library:**
   - Continue with custom bitmap font renderer?
   - Or integrate Pillow/PIL for more flexible text/graphics?
   - Or use Qt’s QPainter to render to pixel buffer, then send to hardware?

2. **Brightness Mapping:**
   - Linear brightness (0-100% → PWM 0-100%)?
   - Or gamma-corrected (perceptual linearity)?
   - Or exponential curve (better low-light perception)?

3. **Error Handling:**
   - If hardware connection fails, fall back to simulator?
   - Or display error and require restart?
   - Or show hardware error on operator console?

4. **Multi-Display Support:**
   - Should operator console and public display be on separate computers?
   - Or continue single-computer architecture?

5. **Configuration Migration:**
   - Keep backward compatibility with config.py?
   - Or enforce database-only configuration?
   - Or provide import/export for config portability?

11. Conclusion & Next Steps

11.1 Summary

The ATLAS Master Flightline Clock is a **well-architected, functional prototype** with solid foundations:
- ✅ Clean MVC separation
- ✅ Database-driven persistence
- ✅ Adaptive 6-row display rendering
- ✅ Professional operator console
- ✅ Settings tab with display selection, brightness control, and system controls **already implemented**

Critical gaps identified:
- ❌ No Interface Control Document (ICD-001) for LED hardware
- ❌ No Hardware Abstraction Layer (only simulator exists)
- ❌ Auto-boot launcher not implemented
- ❌ No physical hardware integration

11.2 Immediate Deliverables

This analysis enables:
1. **Generation of ATLAS-ICD-001** (modular/architectural version, hardware details TBD)
2. **Implementation of HAL** (DisplayBackend abstract class + refactored simulator)
3. **Auto-boot launcher script** for macOS
4. **Settings tab enhancements** (if “LED Module vs LCD Monitor” distinction is truly needed)

11.3 Recommended Actions

For User/Product Owner:
1. Clarify display type dropdown requirements (current implementation vs. desired behavior)
2. Confirm hardware platform for LED panels (Raspberry Pi, controller, network)
3. Approve ICD-001 modular approach (architectural now, hardware specs later)

For Development:
1. Generate ATLAS-ICD-001 document (architectural/generic version)
2. Implement DisplayBackend abstract class and refactor simulator
3. Create macOS LaunchAgent for auto-boot
4. Add backend selector to Settings tab (Simulator, HUB75, Network, Serial)

For Hardware Procurement:
1. Specify LED panel requirements (P5/P10, indoor/outdoor, dimensions)
2. Select controller/driver approach (direct GPIO, serial, network)
3. Finalize ICD-001 with actual hardware specifications post-procurement

---

Document Status: Complete
Next Document: ATLAS-ICD-001 (Interface Control Document)
Author: Abacus AI Agent
Review Date: July 3, 2026